

Deep Dive — Selecting, Migrating, and Maintaining a Model Choice

Part 11 —  The LLM Model Landscape · 11-llm-models/notes/02-selecting-and-migrating-models.md

Section 1-6 of notes/01 [11-llm-models/notes/01-the-model-landscape.md] gave you the axes. This file gives you the process: how to actually run a selection, how to swap a model in production without breaking it, and how to keep your choice correct after you've shipped — because you will need to re-decide this every few months whether you plan to or not.

1. The selection framework: constraints first, leaderboard never

The single most common mistake in model selection is starting with "which model is best" instead of "what does this task actually require." Do it in this order, every time:

1. TASK PROFILE (write this down BEFORE looking at any model)
 - └─ Accuracy bar -- what error rate is tolerable? Who is harmed by a wrong answer?
 - └─ Latency budget -- p50/p95 acceptable response time, sync vs. async
 - └─ Cost budget -- \$/request ceiling at your expected volume
 - └─ Compliance / data-residency constraints -- can data leave your perimeter? Which jurisdictions?
2. SHORTLIST candidates using public benchmarks + tier fit (notes/01 section 1 and 3)
 - this is the ONLY legitimate use of a public leaderboard: narrowing 30 models to 4-5
3. SCORE the shortlist against YOUR eval set (Module 15), not the public benchmark
 - run code/model_selection_scorer.py with your real numbers, weighted by what THIS task cares about
4. PILOT the top 1-2 candidates on live-adjacent traffic before full rollout
 - shadow traffic or a small percentage rollout, measured the same way as step 3
5. DECIDE, PIN THE VERSION, and write down why -- future-you will re-litigate this in 3 months

Steps 1 and 3 are the ones people skip, and skipping them is why "just use whatever's on top of the leaderboard" keeps producing expensive, avoidable mistakes: the leaderboard doesn't know your latency budget, your compliance constraints, or what your actual task distribution looks like.

`code/model_selection_scorer.py` in this module implements steps 1 and 3 as a runnable, transparent weighted-criteria tool — plug in your real task profile and real candidate specs, and it shows its reasoning, not just a number.

A concrete failure this order prevents: a team picks the frontier model because it scores highest on a public reasoning benchmark, ships it, and only then discovers their latency budget was 800ms p95 and the frontier model's typical response time is 4 seconds. Define the constraint first and half your candidates never make the shortlist.

2. Cost at scale and the build-vs-buy decision

Selection doesn't stop at "which model" — it includes "hosted API or self-hosted weights," which is really a volume and control question, not a capability question.

- Run Module 04's cost calculator [04-llm/code/llm_cost_calculator.py] with your actual workload shape first. It will tell you the per-request and monthly cost at each provider tier, and it computes a rough self-hosting break-even volume from GPU-hour cost, utilization, and throughput assumptions.
 - The two numbers that decide build-vs-buy are **sustained monthly volume** and **realistic utilization** — a GPU idling overnight is the most common reason self-hosting loses on paper math that looked favorable.
 - Module 16's AWS Bedrock track [16-ai-tech-stack/tracks/aws-bedrock] is the practical middle ground: a managed host for both proprietary and open-weights models, letting you avoid owning GPU fleets while still choosing an open-weights model for data-residency or licensing reasons.
 - Compliance constraints can override a pure cost calculation: if data cannot leave a jurisdiction or a network perimeter at all, self-hosting or a private-VPC deployment may be the only compliant option regardless of what the break-even math says. Check Module 14 [14-ai-compliance] before the cost model, not after — a cheaper option that's non-compliant isn't actually an option.
-

3. Multi-model strategy: don't marry one provider

Every model choice you make today will be wrong eventually — not because you chose badly, but because the landscape moves. The fix isn't picking better; it's architecting so a change of mind is cheap.

The model gateway pattern (Module 10 [10-ai-architecture]): put one abstraction layer between your application code and every model call. The application asks for "a mid-tier chat completion" or "a fast classification"; the gateway resolves that to a specific provider and model version, handles retries, fallback to a secondary provider on outage, and logs every call uniformly for Module 15's eval pipeline. This is the same idea as a database access layer — you don't want raw provider SDK calls scattered through business logic any more than you want raw SQL scattered through it.

What actually breaks when you swap the underlying model

Swapping a model is never just "point the client at a new endpoint." These are the concrete failure modes, in the order teams usually discover them (the hard way):

What breaks	Why	What to check before swapping
Prompt sensitivity	The exact wording, delimiter style, and few-shot format a prompt was tuned against is specific to one model's training	Re-run your full eval set on the new model with the <i>unmodified</i> prompt before touching it
Tool-calling format drift	Function-calling schemas, parallel tool-call support, and how models signal "call vs. respond" differ across providers and even across versions of the same provider	Test every tool definition your app uses; check for parallel-call behavior changes
Default verbosity/style	Models differ in default response length, hedging language, and markdown formatting habits, which changes downstream parsing and user-facing tone	Diff a sample of outputs side by side, not just pass/fail on your eval
Refusal boundaries	Safety tuning differs across providers and versions; a prompt that worked can start triggering refusals, or a previously-refused prompt can start succeeding	Include boundary/edge cases in your eval set explicitly
Structured-output guarantees	"JSON mode" and schema-constrained decoding are implemented differently (true grammar constraint vs. best-effort prompting) across providers	Verify schema validity rate empirically, don't assume parity
Context-handling behavior	Different lost-in-the-middle profiles and context-rot sensitivity at the same nominal window size	Re-run any long-context task through your own eval, not the vendor's needle test

A practical migration checklist

1. Freeze the current model's outputs on your full eval set as a baseline (Module 15).
2. Run the identical eval set, identical prompts, against the candidate model. No prompt tuning yet.
3. Diff pass rates *and* look at raw outputs for the categories above — a stable pass rate can hide a verbosity or tone shift that a downstream consumer (a UI, a parser, a human reviewer) will notice.
4. Only now, tune the prompt for the new model if needed, and re-run the eval to confirm the tuned version still beats the baseline you started with.
5. Shadow-deploy: send the new model live traffic without serving its response, compare offline.
6. Canary a small percentage of real traffic, with automatic rollback wired to your eval/monitoring signals.
7. Roll forward, keep the previous model+version configured as a documented fallback, not deleted.

4. Versioning and deprecation: an ongoing obligation, not a one-time decision

Providers deprecate model versions on a published (and sometimes unpublished) timeline. Treat "which model are we using" as a line item you revisit on a schedule, not a decision you make once at launch.

- **Pin exact model versions in production**, not a `-latest` or aliased name. An alias that silently points to a new version under you is a model swap you didn't choose to make, didn't test, and won't notice until something downstream breaks. The same prompt-sensitivity, tool-calling, and verbosity risks from section 3 apply — except now they happen to you without warning instead of on a schedule you control.

- **Your eval suite is what catches a silent regression.** Run it on a schedule (Module 15) against your pinned version to confirm nothing changed, and run it again before ever moving to a new pinned version. If you don't have a suite, you have no way to distinguish "the provider improved the model" from "the provider broke something you depend on" until a user complains.
 - **Track provider deprecation calendars deliberately.** Every major provider publishes a deprecation date for old model versions with a mandatory migration window (commonly a matter of months). Put those dates in a calendar with an owner, the same way you'd track a certificate expiry or a database EOL — an LLM dependency is an infrastructure dependency now, not a one-off integration.
 - **Budget migration time as a recurring cost,** not a surprise project. A team running three or four model-dependent features should expect to run the migration checklist above at least once or twice a year per feature, even if they never wanted to change providers.
-

5. Snapshot: frontier families, as of September 2026

This table is a point-in-time reference, not a fact you should still trust by the time you read this. Model names, prices, and context windows in this space change on a timescale of weeks. Verify every cell against the linked official page before using it to make a decision — the table's only job is to show you *what kind of question to ask*, not to answer it for you.

Family	Typical strength / fit	Where to check current specs
Anthropic Claude (Opus 5, Sonnet 5, Haiku 4.5, Fable 5.1 as of this writing)	Agentic coding and long-running tool-use workflows; consistent tool-calling behavior across the tier line	https://claude.com/pricing · https://docs.claude.com/
OpenAI GPT	Broad general-purpose capability, large ecosystem/tooling, frequent reasoning-tier releases	https://openai.com/api/pricing/ · https://platform.openai.com/docs/models
Google Gemini	Very large context windows across the entire tier line (including the cheapest tier), tight integration with Google Cloud/Workspace data	https://ai.google.dev/pricing · https://ai.google.dev/gemini-api/docs/models
Meta Llama	Open-weights, large community and tooling ecosystem, natively multimodal in recent releases; Meta's roadmap for the family has shifted meaningfully across 2025-2026 — verify what's current	https://ai.meta.com/llama/
Mistral	Strong open-and-commercial hybrid strategy from a European lab; relevant by default when EU data-sovereignty rules shape your provider list	https://mistral.ai/products/la-plateforme · https://docs.mistral.ai/
DeepSeek (open-weights)	Frontier-competitive open-weights MoE models at aggressive pricing; a default shortlist entry when self-hosting or data residency is a hard requirement	https://api-docs.deepseek.com/quick_start/pricing/
Qwen / Alibaba (open-weights)	Broad size range (sub-1B to trillion-parameter MoE) under a permissive license, strong multilingual coverage	https://qwenlm.github.io/ · https://www.alibabacloud.com/help/en/model-studio/

Independent, frequently-updated cross-provider comparisons (price/latency/quality curves, not vendor marketing) are worth bookmarking over any static table: see [resources/RESOURCES.md](#) [11-11m-models/resources/RESOURCES.md].

6. Forward links

Idea here	Where it returns
Your own eval set as the real scoring mechanism	Module 15 [15-ai-evals]
Model gateway, fallback, retries	Module 10 [10-ai-architecture]
License review as part of a build-vs-buy decision	Module 14 [14-ai-compliance]
Managed hosting for open-weights models	Module 16 AWS Bedrock track [16-ai-tech-stack/tracks/aws-bedrock]
Cost modeling mechanics	Module 04 [04-llm]
Deprecation and version pinning as a governance artifact	Module 12 [12-ai-governance]